

A Smart Contract Vulnerability Detection Manner Based on Large Language Model

I-Fang Su[1(✉)], Shun-Ming Wang[2], Yu-Chi Chung[2], and Yi-Hsien Tsai[2]

[1] National University of Tainan, Tainan 900392, Taiwan
ifang.su@gmail.com
[2] National Kaohsiung University of Science and Technology, Kaohsiung 807618, Taiwan
ycchung@nkust.edu.tw

Abstract. In this research, we introduce an advanced approach for the detection of smart contract vulnerabilities leveraging Large Language Models (LLMs). Smart contracts are pivotal in the ecosystem of decentralized finance (DeFi), functioning as automated protocols for data management and transaction execution. The foundation of numerous blockchain-based applications lies in smart contract technology. Nevertheless, these contracts' code vulnerabilities can become targets for malicious exploitation, leading to substantial financial damages, exemplified by the 2016 Dao smart contract incident which incurred a loss of 55 million USD. In response to such challenges, detection mechanisms for smart contract vulnerabilities have been devised, drawing upon conventional static analysis, fuzzy testing, and machine learning methodologies. Owing to the swift progression of LLMs, such as GPT, a broad spectrum of entities has adopted these models for routine operational management. By recognizing LLMs' inherent capability to comprehend programming code, we investigate their aptitude for identifying smart contract vulnerabilities. We have integrated prompt engineering techniques, including the Chain of Thought (CoT), Plan-and-Solve, and few-shot learning, to augment the LLMs' vulnerability detection efficacy. Furthermore, a sequence of empirical studies has been orchestrated to validate the effectiveness of our proposed prompt engineering strategies against diverse smart contract vulnerabilities.

Keywords: detection · large language model · GPT · prompt engineering

1 Introduction

Decentralized finance (DeFi) has emerged as a transformative force in the financial industry [1], offering a compelling alternative to traditional financial systems. By leveraging blockchain technology, DeFi eliminates the need for intermediaries such as banks and brokerages, enabling direct peer-to-peer financial transactions. This innovative approach has gained widespread traction due to its numerous advantages, including enhanced accessibility, transparency, automation, and security [2, 3].

At the heart of DeFi, lies the concept of decentralization, which removes the control of financial activities from centralized institutions and empowers individuals to manage

© The Author(s) 2026
T.-H. Meen et al. (Eds.): ICKII 2024, LNEE 1481, pp. 16–25, 2026.
https://doi.org/10.1007/978-981-95-2113-5_2

their finances autonomously. This is achieved through the use of blockchain technology which creates a distributed and immutable ledger of transactions [4]. This decentralized nature fosters transparency and trust among participants, as all transactions are publicly viewable and verifiable.

DeFi uses blockchain technology to provide a secure and immutable record of transactions [5]. This technology, such as underpinning cryptocurrencies, is applied as a distributed database stored across a network of nodes (computers, phones, tablets, etc.) in a decentralized manner. Each block in the blockchain contains a set of transactions and a unique hash code, linking it to the previous block and forming a chain.

The most crucial feature of blockchain is its immutability [6]. Once a transaction is recorded in a block, it is replicated across all nodes in the network. Any attempt to tamper with a block's data would result in a different hash code, alerting the network to unauthorized modifications. Smart contracts, introduced by Nick Szabo in 1994, are self-executing agreements encoded into the blockchain. Smart contracts are a set of agreements that are defined digitally and that contain information about how the participants will fulfill those agreements [7]. They automate complex transactions based on predefined conditions, eliminating intermediaries and enhancing efficiency, transparency, and security [3].

Despite the immense potential of smart contracts, the prevailing market conditions, human nature, and the inherent value proposition of cryptocurrencies have driven a profit-oriented mindset, even in non-financial applications. Financial projects, on the other hand, can hold crypto assets with market capitalizations ranging from millions to billions of dollars. Given that smart contracts execute based on code, and their source code is often publicly accessible on the blockchain, the DeFi Summer allows for the emergence of numerous projects hastily adopting the source code of existing projects. This lack of innovation and vulnerability patching exposed the DeFi Summer ecosystem to a surge of hacker attacks. Consequently, the demand for more sophisticated smart contract vulnerability detection systems has become increasingly urgent.

In this study, we design a smart contract code vulnerability detection system (the terms "vulnerability" and "weakness" are used interchangeably in this study to avoid monotony). As previously mentioned, hackers easily exploit vulnerabilities in smart contracts, leading to significant financial losses. Traditional methods rely on static analysis [8], which depends on expert-defined fixed rules to detect syntactic vulnerabilities. However, due to the lack of semantic analysis capabilities, static analysis methods may lead to false positives and false negatives.

With the development of deep learning, many methods such as abstract syntax tree (AST), control flow graph (CFG), program dependency graph (PDG), attention mechanism, and pooling layer, etc. are used to detect smart contract vulnerabilities. The main reason for using deep learning is that when the number of parameters in a language model exceeds a threshold, the LLM triggers the "emergent phenomenon" [8], as shown in Fig. 1. The axis in the figure is the scale of the model, and the more to the right, the more parameters the model has. The axis is the evaluation metrics of different natural language tasks. The red dashed line in the figure represents a random phenomenon, which means that the model's performance on the task is random, good or bad, and does not improve as the number of parameters increases. However, in most tasks, when the

scale of the model exceeds a certain threshold, the model's performance on the task shows a leapfrog growth, and it breaks away from the random phenomenon. The more parameters the model has, the better the performance. It is also because of this emerging phenomenon that in recent years, people have become increasingly confident in the performance of large language models.

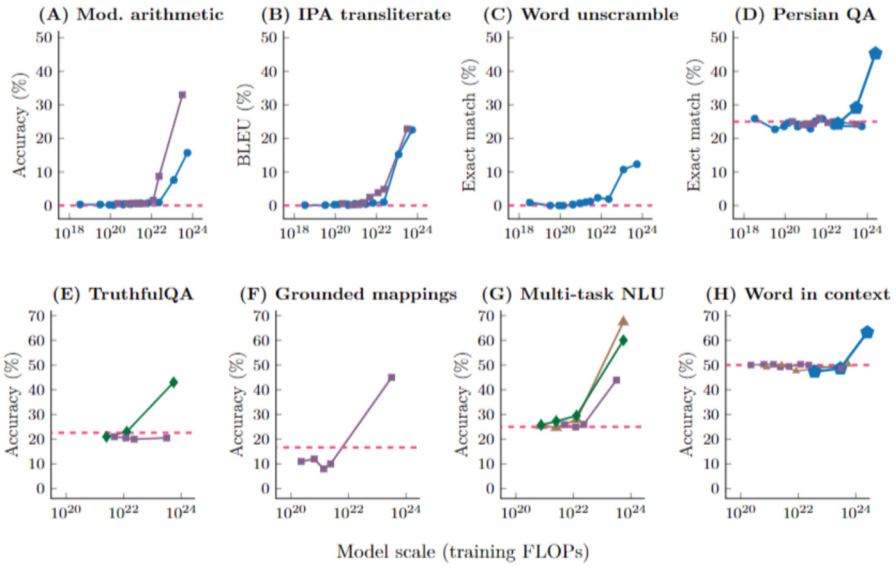


Fig. 1. Emergent abilities of LLM [9].

We verify the ability of LLMs to "understand" code. In the study, we collect four common vulnerabilities in smart contracts, namely re-entry vulnerabilities, integer overflow (including overflow and underflow) vulnerabilities, and infinite loop vulnerabilities. We let the LLM model read these codes and determine whether there are vulnerabilities in the code. The LLMs we used to be OpenAI's GPT 3.5 and GPT 4 [1].

Although the model can understand language, it is still incapable of handling particular tasks (such as reading legal documents). Therefore, we use fine-tuning to make large language models perform better on these tasks. We collectively refer to these special tasks as downstream tasks. In this study, detecting vulnerabilities in smart contracts is the downstream task.

For fine-tuning, we input the data of the downstream task into the language model, adjust the parameters of the language model through backpropagation of the prediction error, and use the gradient descent mechanism to gradually improve the performance of the language model on the downstream task. This pre-train and fine-tune mode has been popular since 2016, and in many related studies, this mechanism has been used to deal with natural language tasks and achieved excellent results [10].

In addition to exploring the ability of large language models to detect smart contract vulnerabilities, we also consider the performance of different prompt engineering mechanisms. In the study, we explore the Chain of Thought's (CoT) performance [13], Prompt

assembling [9, 11, 12], and few-shot learning on detecting contract vulnerabilities. This is the first study to explore how to use large language models to detect smart contract vulnerabilities. As smart contracts become popular and DeFi grows, vulnerability issues in smart contracts become increasingly serious as they lead to a large number of financial attacks. Once this happens, there is a huge impact is on the development of blockchain and decentralized finance. Therefore, it is imperative to develop efficient mechanisms to detect smart contract vulnerabilities. Applying large language models to detect smart contract vulnerabilities allows for the following benefits: (1) a mechanism for using large language models to "understand" code to expand the application range of large language models, and (2) detection mechanisms with better prevention capabilities for unknown vulnerabilities since large language models understand code to catch smart contract vulnerabilities. Therefore, the result of this study indicates the potential to further improve the detection success rate.

2 Related Work

Large language models (LLMs) have garnered significant attention due to their remarkable capabilities. However, the massive number of parameters poses challenges in fine-tuning and developing prompt engineering techniques to improve LLM performance. As illustrated in Table 1, prompt engineering involves incorporating detailed task descriptions into LLM prompts, guiding them toward specific task completion.

Table 1. Parameters Used in LLM

Name	Release date	Number of parameters
BERT	2018	340 million
XLNet	2019	340 million
T5-base	2020	223 million
T5-larget	2020	783 million
GPT-2	2019	1.5 billion
GPT-3	2020	175 billion
Megatron-Turing	2021	530 billion
Claude	2021	52 billion
Bloom	2022	175 billion
LLAMA	2023	65 billion
LLAMA 2	2023	70 billion

The concept of prompt engineering was initially introduced in GPT-3, where OpenAI referred to it as "in-context learning." This approach involves embedding task requirements within the prompt and instructing the LLM to adhere to these specifications. OpenAI's official documentation [14] elaborates on various techniques for enhancing

the performance of GPT models in downstream tasks. These techniques are collectively termed prompt engineering. Since 2023, researchers have explored prompt engineering techniques to boost language model performance.

One notable example is the CoT mechanism proposed by Kojima et al. in 2023 [15]. The authors discovered that simply adding the instruction "Let's think step by step" to the prompt significantly improved the model's task performance. Wei et al. [13] extended Kojima et al.'s work by conducting experiments in few-shot learning to demonstrate the effectiveness of few-shot CoT in enhancing model performance. However, both zero-shot and few-shot CoTs require manual prompt construction by users. Different tasks demand different examples to achieve optimal model performance, limiting the scalability of this approach. Moreover, constructing effective examples often necessitates domain knowledge, which may not be readily available to all users. Wang et al. addressed this issue by introducing Plan-and-Solve Prompting [16]. In this approach, users simply instruct the LLM to identify the problem at hand, devise a solution plan, and then execute the plan to solve the problem. This method eliminates the need for domain knowledge in prompt construction while effectively improving model performance.

Prompt engineering is a powerful tool for enhancing the performance of LLMs in various downstream tasks. By carefully crafting prompts that guide LLMs for specific goals, researchers and practitioners can unlock the full potential of these powerful language models.

3 Smart Contract Vulnerability Detection

The smart contract vulnerabilities are taken from the SmartBugs Dataset-Wild dataset [17] which contains 47,587 smart contract vulnerabilities of Ethereum solidity. We use the code of the weaknesses of Infinite Loop, Integer Overflow/Underflow, and Reentrancy as a dataset to verify our method. These four weaknesses will be explained below.

3.1 Infinite Loop

An infinite loop occurs when a smart contract is stuck in a continuous loop, repeatedly executing the same code without any progress. This causes severe consequences, such as excessive gas fees and rendering the contract unusable.

3.2 Integer Overflow or Underflow

Integer overflow or underflow happens when a mathematical operation in a smart contract results in a value that exceeds the integer variable's maximum or minimum representable value. This leads to incorrect calculations and unexpected behavior, potentially allowing attackers to manipulate the contract's execution.

3.3 Reentrancy

Reentrancy vulnerability arises when a function is called multiple times before the first call has completed its execution. This allows malicious actors to exploit the race condition

and repeatedly extract funds from a smart contract, often referred to as a flash loan attack. We employ the text-davinci-003 model from OpenAI, an LLM based on GPT-3.5 and designed for instruction-based tasks. Unlike chat-based LLMs that handle multiple conversational turns, instruction-based LLMs operate on single prompts.

We evaluate the performance of zero-shot and few-shot prompts to determine the optimal prompt strategy. Zero-shot learning involves no examples in the prompt, requiring the model to understand the instructions and analyze the code for vulnerabilities. Few-shot learning, on the other hand, provides a few examples in the prompt to guide the model and potentially enhance its performance.

For each of the four vulnerability types (infinite loop, integer overflow/underflow, and reentrancy attack), we prepared 300 test cases, half of which contain vulnerabilities and the other half are vulnerability-free. These test cases are fed into the LLM, which is tasked with the presence of specific vulnerabilities in the code. This translates into a binary classification task, determining whether a given code snippet contains a vulnerability or not.

To enhance the model's performance for each vulnerability type, we employ three prompt engineering techniques within the zero-shot setting: Baseline prompt, Chain-of-Thought (CoT) [15], and Plan-and-Solve [16]. Additionally, we experimented with few-shot learning, incorporating examples into the prompt to mimic our code analysis process and potentially improve the model's vulnerability detection capabilities.

4 Simulations

We employed four solidity vulnerabilities from the Smartbug Dataset-Wild dataset [17] to evaluate the effectiveness of large language models for vulnerability detection using the GPT-3.5 model from OpenAI. Figure 2 illustrates the distribution of the four vulnerability types in the test set. Each vulnerability dataset contains code snippets with and without the corresponding vulnerability. The model analyzes the code and identifying the presence of the specific vulnerability.

We utilized the text-davinci-003 model from OpenAI, a large language model (LLM) based on GPT-3.5 and designed for instruction-based tasks. According to OpenAI's documentation, text-davinci-003's training data extends only up to 2021, excluding knowledge from subsequent years. The following parameters were employed in our experiments: (1). temperature: 0.1, (2). top-k: 1, and (3). max_tokens: 500.

The temperature parameter controls the "creativity" of the LLM's generated results. A value of 0 indicates that the LLM strictly adheres to its training data. Higher temperature values introduce more creativity. In the experiments, we set this value to 0.1. Top-k and temperature are used to control the LLM's creativity. OpenAI's developer documentation suggests setting either temperature or top-k, and we opted for the default top-k value. Max_tokens determines the number of tokens in the LLM's response. Higher values result in longer responses.

Bold values in Table 2 indicate the best performance. Table 2 shows simple prompt engineering techniques such as "Let's think step by step" significantly improve performance compared to the baseline scenario without prompt engineering. For instance, in the infinite loop case, the accuracy increases from 61 to 67%, while for reentrancy, the

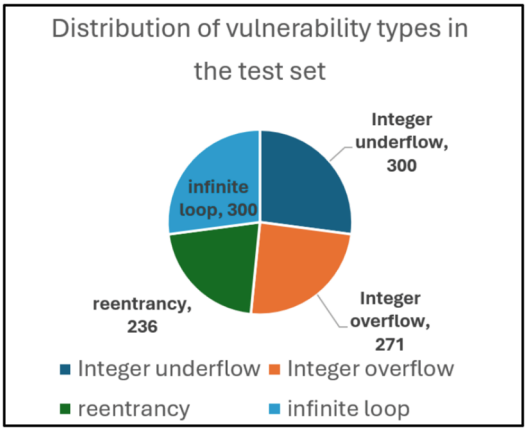


Fig. 2. Distribution of vulnerability types in test set.

improvement is from 57.84 to 66.76%. Plan-and-Solve generally outperforms CoT, suggesting that having the LLM carefully consider the problem's essence and independently derive a solution can extract more relevant knowledge from the LLM to address the issue. Interestingly, Plan-and-Solve with user-specified instructions yielded surprisingly less satisfactory results. This is likely due to the introduction of noise from user-specified instructions, hindering the model's reasoning ability.

Table 2. Prediction Scores for Different Vulnerabilities Under Zero-Shot Conditions With Varying Prompt Engineering Techniques

Methods	Infinite loop	Integer overflow	Integer underflow	Reentrancy
Baseline	61%	67.89%	49.28%	57.84%
CoT	**67%**	64.94%	50.39%	66.76%
Plan-and-Solve	61%	**67.52%**	**52.94%**	**67.07%**
Plan-and-Solve with user-specified instruction	58%	65.31%	51.75%	64.00%

Table 3 presents the prediction scores for different vulnerabilities under few-shot conditions with varying prompt engineering techniques. Few-shot learning involves providing examples in the prompt to guide the model in identifying code vulnerabilities. Due to the text-davinci-003 model's maximum token limit of 4096, we could only include one example in the prompt.

Few-shot learning outperforms zero-shot in most cases, even exceeding zero-shot by nearly 20% in certain situations (integer overflow). This indicates that providing examples can indeed enhance the model's performance. However, few-shot learning occasionally underperforms zero-shot, as observed in the infinite loop case. Regardless

Table 3. Prediction Values of Different Prompt Engineering under Few-Shot Learning

Methods	Infinite loop	Integer overflow	Integer underflow	Reentrancy
Baseline	52%	53.30%	43.33%	47.03%
CoT	**55.66%**	73.43%	55.39%	68.22%
Plan-and-Solve	52%	67.78%	48.04%	**71.15%**
Plan-and-Solve with user-specified instruction	51%	**84%**	**60.06%**	55.55%

of the prompt engineering method, few-shot consistently falls behind zero-shot. This is attributed to the specific examples provided. The model imitates the provided examples to detect vulnerabilities. If the example closely resembles the target vulnerability, the model effectively mimics human-provided examples and achieves favorable results. However, if the example bears minimal resemblance to the target code, the model may become confused, leading to suboptimal performance. In infinite loops, diverse manifestation patterns exist. With only one example provided, the model may struggle when the example's vulnerability manifestation differs from the target code.

The experiment results show that LLM detects weaknesses in the code to a certain extent, and the prompt engineering mechanism effectively improves the model's performance. With examples for model reference, we have improved the model's performance.

5 Conclusion

In this study, we explored the issue of using LLM to detect vulnerabilities in smart contracts. The strong language understanding capabilities of LLM allow for analyzing vulnerabilities in code. In this study, we have adopted OpenAI's GPT-3.5 as the main model and introduced prompt engineering techniques to enhance the model's accuracy in detecting vulnerabilities. We collected four types of vulnerability tests from the smartbug-dataset wild dataset and verified the model's performance in zero-shot and few-shot scenarios. Based on the result of this study, many issues can be addressed. The model used in this study is text-davinci-003 from GPT-3.5. This model can handle less context (only 4096 tokens), and its performance will decrease for longer code. In addition, the knowledge base of text-davinci-003 is only up to 2021, which may not detect newer vulnerabilities. There are more LLMs available, such as Meta's LLaMA2, OpenAI's GPT-4, and Google's PaLM. The performance of these LLMs in detecting vulnerabilities is worth investigating. The mechanism of prompt engineering effectively improves the model's detection performance, so continuously exploring the efficiency of different prompt engineering mechanisms is needed. The Retrieval-augmented generation (RAG) also improves performance. The prompts used in this study are fixed, and the same prompt is used regardless of the code, which may affect the detection performance. Using RAG, the optimal prompt for different codes is configured, which has the potential to further improve performance. In addition to prompt engineering, fine-tuning is

also necessary. By efficiently fine-tuning the model, the model's vulnerability detection performance can be improved.

Acknowledgement. This research was supported by the Ministry of Science and Technology of Taiwan (R.O.C.) under Grants NSTC 113-2221-E-992-077 – and NSTC 113-2221-E-153-004 -.

References

1. Weingärtner, T., Fasser, F., da Costa, P., Farkas, W.: Deciphering DeFi: a comprehensive analysis and visualization of risks in decentralized finance. J. Risk Financ. Manag. **16**, 454 (2023). https://doi.org/10.3390/jrfm16100454
2. Sun, X., Chen, X., Stasinakis, C., Sermpinis, G.: Multiparty democracy in decentralized autonomous organization (dao): evidence from makerdao. SSRN Electron. J. (2022). https://doi.org/10.2139/ssrn.4253868
3. Schär, F.: Decentralized finance: on blockchain- and smart contract-based financial markets. Review. **103**(2), 74 (2021). https://doi.org/10.20955/r.103.153-74
4. Ozili, P.K.: Decentralised finance and cryptocurrency activity in Africa. In: The New Digital Era: Digitalisation, Emerging Risks and Opportunities, pp. 3–11 (2022). https://doi.org/10.1108/s1569-375920220001090a001
5. Yli-Huumo, J., Ko, D., Choi, S., Park, S., Smolander, K.: Where is current research on blockchain technology?—a systematic review. PLoS One. **11**(10), e0163477 (2016). https://doi.org/10.1371/journal.pone.0163477
6. Rawlins, C.H., Jagannathan, S.: An intelligent distributed ledger construction algorithm for IoT. IEEE Access. **10**, 10838–10851 (2022). https://doi.org/10.1109/access.2022.3146343
7. Wang, S., Ouyang, L., Yuan, Y., Ni, X., Han, X., Wang, F.-Y.: Blockchain-enabled smart contracts: architecture, applications, and future trends. IEEE Trans. Syst. Man. Cybern. Syst. **49**(11), 2266–2277 (2019)
8. Wei, J., et al.: Emergent abilities of large language models. In: Transactions on Machine Learning Research (2022)
9. Ding, N., et al.: Open Prompt: an open-source framework for prompt-learning. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics, pp. 105–113 (2021). https://doi.org/10.18653/v1/2022.acl-demo.10
10. Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G.: Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing. ACM Comput. Surv. **55**(9), 815 (2023). https://doi.org/10.1145/3560815
11. Yang, X., Cheng, W., Zhao, X., Yu, W., Petzold, L., Chen, H.: Dynamic prompting: a unified Framework for prompt tuning. https://arxiv.org/abs/2303.02909v2 Accessed 17 Oct. 2023 (2023)
12. Shin, T., Razeghi, Y., Logan, R.L., Wallace, E., Singh, S.: AutoPrompt: eliciting knowledge from language models with automatically generated prompts. In: EMNLP 2020–2020 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference, pp. 4222–4235 (2020). https://doi.org/10.18653/v1/2020.emnlp-main.346
13. Wei, J., et al.: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, vol. 35 (2022). https://arxiv.org/abs/2201.11903v6
14. OpenAI. OpenAI Prompt Engineering Guide (2023). https://platform.openai.com/docs/guides/prompt-engineering. Accessed 08 Nov 2023
15. Kojima, T., Gu, S.S., Research, M.R.G., Matsuo, Y., Iwasawa, Y.: Large language models are zero-shot reasoners. Adv. Neural Inf. Process. Syst. **35**, 22199–22213 (2022)

16. Wang, L., et al.: Plan-and-solve prompting: improving zero-shot chain-of-thought reasoning by large language models, pp. 2609–2634 (2023). https://doi.org/10.18653/v1/2023.acl-long.147
17. Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P.: Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In: Proceedings-International Conference on Software Engineering, pp. 530–541 (2019). https://doi.org/10.1145/3377811.3380364

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

